Programmeeropdrachten Java

Deel I
Klimaatregeling

1 Situatie

Je programmeert de klimaatregeling van een afgesloten ruimte. De ruimte heeft een **raam** en een **deur**, die beide automatisch kunnen worden aangestuurd. Twee sensoren leveren meetwaarden:

- temperatuur in graden Celsius;
- relatieve luchtvochtigheid in procenten.

De gebruiker voert de sensorwaarden als gehele getallen in. Het programma bepaalt vervolgens hoe ver het raam en de deur geopend moeten worden.

2 Regels voor het raam

Het raam reageert op de temperatuur:

Temperatuur	**Stand raam**
Lager dan of gelijk aan 25 °C	0% open
Hoger dan 25 °C en lager dan 35 °C	lineair tussen 0% en 100% open
35 °C of hoger	100% open

Bij een temperatuur van 26 °C staat het raam dus 10% open. Bij 30 °C staat het raam 50% open. Bij 35 °C staat het raam 100% open.

Gebruik voor het tussenliggende gebied deze formule:

```
1         percentageRaam = (temperatuur - 25) * 10
2
```

3 Regels voor de deur

De deur reageert op de luchtvochtigheid:

Luchtvochtigheid	**Stand deur**
Lager dan 60%	0% open
Vanaf 60% tot en met 80%	50% open
Hoger dan 80%	100% open

4 Invoercontrole

Een geldige temperatuur ligt tussen −20 °C en 60 °C, inclusief de grenswaarden. Een geldige luchtvochtigheid ligt tussen 0% en 100%, inclusief de grenswaarden.

Als een invoer ongeldig is, meldt het programma welke waarde fout is en vraagt het die waarde opnieuw. Bereken de standen pas wanneer beide invoerwaarden geldig zijn.

5 Verplichte methoden

Neem ten minste de volgende methoden op. Je mag extra methoden toevoegen.

```
1        public static boolean isGeldigeTemperatuur(int temperatuur)
2        public static boolean isGeldigeLuchtvochtigheid(int luchtvochtigheid)
3        public static int bepaalRaamStand(int temperatuur)
4        public static int bepaalDeurStand(int luchtvochtigheid)
5
```

5.1 Betekenis van de methoden

- `isGeldigeTemperatuur` ontvangt een temperatuur en retourneert `true` of `false`.
- `isGeldigeLuchtvochtigheid` ontvangt een luchtvochtigheid en retourneert `true` of `false`.
- `bepaalRaamStand` ontvangt een geldige temperatuur en retourneert een geheel openingspercentage van 0 tot en met 100.
- `bepaalDeurStand` ontvangt een geldige luchtvochtigheid en retourneert 0, 50 of 100.

De methode `main` roept deze methoden aan en toont de resultaten. `main` bevat dus niet zelf alle beslisregels.

Hieronder wordt een flowchart gegeven van het programma.

6 Opdracht

Maak een Java-programma dat:

1. Begin met het inlezen van de temperatuur doe dit net zo lang tot er een geldige temperatuur is ingelezen. Gebruik hier de template zoals in listing 1 wordt weergegeven.
2. Lees de luchtvochtigheid in en doe dit net zolang totdat een geldige luchtvochtigheid is ingelezen.
3. de openingspercentages van raam en deur berekent;
4. de uitkomsten duidelijk op het scherm toont;
5. voor de berekeningen **eigen methoden met parameters en returnwaarden** gebruikt.

Gebruik alleen `int` voor de sensorwaarden. Gebruik geen globale variabelen. Waarden die een methode nodig heeft, geef je mee als parameter. Een methode die een berekening uitvoert, geeft het resultaat terug met `return`.

```
1
2  import java.util.Scanner;
3
4  public class Main {
5
6      public static boolean isGeldigeTemperatuur(int temperatuur) {
7
8      }
9
10     public static boolean isGeldigeLuchtvochtigheid(int luchtvochtigheid) {
11
12     }
13     public static int bepaalRaamStand(int temperatuur) {
14
15     }
16     public static int bepaalDeurStand(int luchtvochtigheid) {
17
18     }
19
20
21
22     public static void main(String[] args) {
23         Scanner scanner = new Scanner(System.in);
24         int temp = scanner.nextInt();
25         Boolean oke=isGeldigeTemperatuur(temp);
26         while(....) {
27             temp = scanner.nextInt();
28             oke=isGeldigeTemperatuur(temp);
29         }
30         // controleer de luchtvochtigheid
31
32
33         //en de rest
34     }
35 }
```

Listing 1: Een raamwerk van de main class

7 Voorbeeld

```
Voer de temperatuur in (-20 t/m 60): 30
Voer de luchtvochtigheid in (0 t/m 100): 72

Klimaatregeling
Temperatuur: 30 graden Celsius
Luchtvochtigheid: 72%
Raam: 50% open
Deur: 50% open

```

Voorbeeld met ongeldige invoer:

```
Voer de temperatuur in (-20 t/m 60): 70
Ongeldige temperatuur. Voer een waarde tussen -20 en 60 in.
Voer de temperatuur in (-20 t/m 60): 26
Voer de luchtvochtigheid in (0 t/m 100): 110
Ongeldige luchtvochtigheid. Voer een waarde tussen 0 en 100 in.
Voer de luchtvochtigheid in (0 t/m 100): 45

Klimaatregeling
Temperatuur: 26 graden Celsius
Luchtvochtigheid: 45%
Raam: 10% open
Deur: 0% open

```

8 Testgevallen

Test je programma minimaal met onderstaande waarden.

Temperatuur	**Luchtvochtigheid**	**Verwacht raam**	**Verwachte deur**
25	59	0%	0%
26	60	10%	50%
30	80	50%	50%
35	81	100%	100%
−20	0	0%	0%
60	100	100%	100%

Test ook een ongeldige temperatuur en een ongeldige luchtvochtigheid.

Deel II

Hardloopmonitor

Situatie

Tijdens het hardlopen worden elke seconde drie waarden van een persoon gemeten:

- hartslag in slagen per minuut (bpm);
- zuurstofsaturatie in het bloed (SpO_2, in procenten);
- lichaamstemperatuur in hele graden Celsius.

Het programma leest deze metingen in en geeft een advies over het looptempo. De hardloper kan harder lopen, in hetzelfde tempo doorlopen, langzamer lopen of direct stoppen.

Leerdoelen

Na deze opdracht kun je:

- gehele getallen invoeren en valideren;
- `if`/`else if`/`else` gebruiken;
- methoden schrijven met parameters;
- methoden schrijven die een waarde teruggeven met `return`;
- de teruggegeven waarde gebruiken in `main`;
- een programma overzichtelijk opsplitsen in kleine methoden.

Beslisregels

Gebruik onderstaande regels in deze volgorde. Zodra een stopregel geldt, hoeft het programma de overige regels niet meer te beoordelen.

Prioriteit	Voorwaarde	Advies
1	Zuurstofsaturatie lager dan 90%	**STOPPEN**
2	Temperatuur hoger dan 39 graden Celsius	**STOPPEN**
3	Hartslag hoger dan 190 bpm	**STOPPEN**
4	Zuurstofsaturatie van 90% t/m 93%	**LANGZAMER LOPEN**
5	Temperatuur van 39 graden Celsius	**LANGZAMER LOPEN**
6	Hartslag hoger dan 170 bpm t/m 190 bpm	**LANGZAMER LOPEN**
7	Hartslag lager dan 120 bpm, saturatie minimaal 95% en temperatuur maximaal 37 graden Celsius	**HARDER LOPEN**
8	Alle andere geldige situaties	**ZELFDE TEMPO**

Bij `LANGZAMER LOPEN` moet het programma ook aangeven waarom dat advies wordt gegeven. Als meerdere langzamer-regels waar zijn, mag je één reden tonen.

Dit zijn oefenwaarden voor een programmeeropdracht en geen medisch advies.

Opdracht

Schrijf een Java-programma met de klassenaam `HardloopMonitor`.

1. Vraag de gebruiker om de hartslag, zuurstofsaturatie en temperatuur van één meetmoment.
2. Controleer of de invoer binnen een realistisch bereik ligt:
   - hartslag: 30 t/m 240 bpm;
   - zuurstofsaturatie: 70 t/m 100 procent;
   - temperatuur: 30 t/m 43 graden Celsius.
3. Toon bij ongeldige invoer een duidelijke foutmelding en bepaal geen loopadvies.
4. Bepaal bij geldige invoer het juiste advies volgens de tabel.
5. Toon de ingevoerde waarden en het advies duidelijk op het scherm.
6. Bij het advies `LANGZAMER LOPEN` toon je ook de reden.

Gebruik alleen `int` voor de metingen. De temperatuur wordt dus als heel aantal graden ingevoerd.

Verplichte methoden

`main` mag alleen invoer regelen, methoden aanroepen en resultaten afdrukken. De beslislogica staat dus niet volledig in `main`.

Implementeer minimaal deze methoden:

```
1        public static boolean zijnMetingenGeldig(
2        int hartslag, int saturatie, int temperatuur)
3
```

Deze methode ontvangt de drie metingen als parameters en retourneert `true` wanneer **alle** metingen binnen het toegestane bereik vallen; anders `false`.

```
1        public static String bepaalAdvies(
2        int hartslag, int saturatie, int temperatuur)
3
```

Deze methode retourneert precies één van de volgende teksten: `"STOPPEN"`, `"LANGZAMER LOPEN"`, `"HARDER LOPEN"` of `"ZELFDE TEMPO"`.

```
1        public static String bepaalReden(
2        int hartslag, int saturatie, int temperatuur)
3
```

Deze methode retourneert de reden van het advies, bijvoorbeeld `"zuurstofsaturatie te laag"` of `"hartslag te hoog"`. Bij `HARDER LOPEN` en `ZELFDE TEMPO` mag deze methode een lege tekst retourneren.

Voorbeelduitvoer

```
1        Hartslag (bpm): 176
2        Zuurstofsaturatie (%): 97
3        Temperatuur (hele graden C): 37
4
5        --- Meetresultaat ---
6        Hartslag: 176 bpm
7        Zuurstofsaturatie: 97%
```

```
        Temperatuur: 37 C
        Advies: LANGZAMER LOPEN
        Reden: hartslag te hoog

```

Testgevallen

Test je programma ten minste met deze waarden.

Hartslag	Saturatie	Temperatuur	Verwacht advies
110	98	37	HARDER LOPEN
135	97	37	ZELFDE TEMPO
175	97	37	LANGZAMER LOPEN
150	92	37	LANGZAMER LOPEN
150	97	39	LANGZAMER LOPEN
145	89	37	STOPPEN
195	98	37	STOPPEN
150	97	40	STOPPEN
250	97	37	Ongeldige invoer

Eisen

- Gebruik `Scanner` voor de invoer.
- Gebruik alleen `int` voor de meetwaarden.
- Gebruik geen globale variabelen.
- Geef parameters betekenisvolle namen.
- Laat elke methode één duidelijke taak uitvoeren.
- Gebruik `return` in de verplichte methoden.
- Voorzie het programma van korte, nuttige commentaarregels.
- Laat het programma zien.

Extra uitdaging

Breid het programma uit zodat er meerdere meetmomenten achter elkaar kunnen worden ingevoerd. Stop wanneer de gebruiker `0` invoert als hartslag of wanneer het advies `STOPPEN` is. Toon aan het eind hoeveel meetmomenten veilig waren en hoeveel keer er langzamer gelopen moest worden.